Yann STOLL-GEYER
Diego APARICIO
TP1

# Rapport des choix de conception

Ce rapport présente les nouveaux choix de conception réalisés depuis le 1er rapport.

## 1. Architecture Globale : Renforcement du MVC

### 1.1. Passage au Pattern Observer (Architecture Événementielle)

Le changement structurel majeur de cette itération vise à découpler totalement le Modèle de la Vue.

- **Avant :** Le contrôleur (**PlacementController**) modifiait le modèle, puis devait parcourir manuellement la vue pour mettre à jour les images, créant un couplage fort et des risques de désynchronisation.
- **Maintenant :**
  - Le **Modèle (Grid)** est *Observable*. Il définit l'interface **GridListener** et notifie ses abonnés lors d'événements clés (notifyCellUpdated, notifyWeaponImpact).
  - La **Vue (GridView)** est *Observatrice*. Elle implémente **GridListener** et réagit automatiquement aux changements d'état.
- **Détail technique :** L'interface **GridListener** ne gère pas seulement des changements d'état permanent, mais aussi des événements éphémères via des méthodes comme onSonarEffect et onWeaponImpact.
- **Gain :** Suppression des dépendances cycliques. Cela permet à la vue de déclencher des affichages temporaires (chiffre du sonar, impact d'arme) sans modifier de manière permanente les données de la case, tout en assurant que la vue se met à jour seule par réaction.

### 1.2. Métamorphose du GameController (State Machine)

- **Avant :** Une dispersion de la logique métier entre un contrôleur principal et des sous-composants (**GridController**, **GridActionListener**) gérant les événements de bas niveau.
- **Maintenant :** Le **GameController** est devenu un chef d'orchestre de haut niveau gérant les phases de jeu via une machine à états (**GameState** :

SETUP, HUMAN_TURN, etc.). Il agit comme assembleur en instanciant et connectant le Modèle et la Vue (Injection de dépendances).
- **Gain :** Point d'entrée unique facilitant la gestion des tours et la détection de fin de partie.

### 1.3. Découplage via Interfaces de Rappel (Callbacks)

- **Problème :** La Vue devait parfois connaître l'implémentation du Contrôleur pour transmettre les actions utilisateur.
- **Solution :** Introduction d'interfaces sémantiques dans le package controller : **AttackListener** pour les tirs et **ObjectPlacementListener** pour le drag & drop.
- **Gain :** Respect de l'Inversion de Contrôle (IoC). La Vue dépend d'un contrat (interface) et non d'une classe concrète.

## 2. Évolution du Modèle et de la Vue

### 2.1. Généralisation par Polymorphisme (PlaceableView)

- **Avant :** La gestion graphique et le "Drag & Drop" étaient spécifiquement codés pour les bateaux via **ShipView**.
- **Maintenant :** Introduction de la classe abstraite **PlaceableView**. Elle centralise la logique d'interaction (souris, déplacement) et d'affichage. Les classes **ShipView**, **TrapView** et **WeaponView** en héritent pour spécifier leur contenu.
- **Gain :** Factorisation du code d'interaction (DRY). L'ajout de pièges ou d'armes dans l'interface graphique se fait sans dupliquer la logique complexe du Drag & Drop.

### 2.2. Contextualisation des Interactions (Attack & AttackResult)

- **Avant :** Un système de retour simple (booléen ou énumération) pour les tirs.
- **Maintenant :** Création d'objets de contextes riches.
    - **Attack** : Encapsule l'attaquant, la cible, l'arme et les coordonnées, permettant aux pièges (ex: Tornade) de modifier la trajectoire du tir.
    - **AttackResult** : Transporte des résultats complexes (liste d'impacts multiples pour la Bombe, contre-attaque pour le Trou Noir, dénombrement pour le Sonar).
- **Gain :** Gestion propre des effets de bord et des réactions en chaîne sans complexifier le contrôleur.

### 2.3. Séparation Sémantique des États

- **Avant :** Confusion possible entre l'apparence d'une case et le résultat logique d'une action.

- **Maintenant :** Distinction stricte :
    - **CellState** (Vue) : État visuel persistant (ex: BOMB_FOUND, ISLAND_HIT) lu par la **CellView**.
    - **AttackStatus** (Logique) : Résultat événementiel d'une action (ex: ALREADY_VISITED, SONAR_RESULT) traité par le Contrôleur.
- **Gain :** Cohérence visuelle (ex: un scan sonar ne "casse" pas l'affichage de l'eau ou des îles).

### 2.4. Gestion Structurée de l'Armement (Inventory)

- **Avant :** Gestion des armes souvent implicite ou mélangée aux attributs du joueur.
- **Maintenant :** Création de la classe **Inventory** qui encapsule la liste des armes disponibles et leurs munitions. Le **Player** possède une instance de cet inventaire pour déléguer la gestion du stock.
- **Gain :** Meilleure encapsulation et préparation pour de futures extensions (ex : ajout de bonus ou rechargement de munitions).

## 3. Optimisation et Expérience Utilisateur (UI/UX)

### 3.1. Gestion Centralisée des Ressources (ImageUtils)

- **Avant :** Gestion des images spécifique aux bateaux (**ShipImageProvider**), peu flexible.
- **Maintenant :** Utilisation d'une classe utilitaire statique **ImageUtils** gérant le chargement sécurisé et la transformation dynamique (rotation, découpe de sprites) pour tous les types d'objets (**Placeable**).
- **Gain :** Code DRY (Don't Repeat Yourself) et meilleure maintenabilité des assets graphiques.

### 3.2. Architecture de Navigation (CardLayout)

- **Avant :** Destruction/Création de fenêtres.
- **Maintenant :** Le **MainFrame** utilise un CardLayout couplé à un dictionnaire de vues et une classe de constantes **ScreenType**.
- **Gain :** Navigation fluide, robuste et performante entre les écrans (Accueil, Config, Jeu, Fin).

### 3.3. Contrôleurs Spécialisés (Config & Stats)

Pour alléger le contrôleur principal, des responsabilités spécifiques ont été déléguées :

- **Configuration :** Le couple **ConfigController** / **GameConfig** valide les paramètres (taille grille, nombre de bateaux) avant le lancement.

- **Statistiques :** Le module **StatsController** / **StatsView** applique le SRP (Single Responsibility Principle) en gérant isolément le calcul et l'affichage des métriques de partie.

### 3.4. Refonte de l'Interaction de Placement

Le **PlacementScreen** délègue strictement :

- Le visuel (Drag & Drop) à **PlaceableView** et JLayeredPane.
- La logique de validation au **PlacementController**.
- **Détail technique :** Le placement aléatoire est sécurisé par un bloc try-catch-finally garantissant que l'interface est nettoyée (méthode clearPlaceableViews) même en cas d'erreur.
- **Gain :** Expérience utilisateur fluide avec feedback visuel immédiat et adaptabilité grâce au **SquarePanel** qui maintient la grille carrée et centrée.